

Unit 11. Decision Trees

1 Cost-Complexity Pruning

ISLR §8.1.1.

Growing until every leaf is nearly pure fits the training data exactly. A threshold on the split gain is not the repair: a split worth little on its own can be the one that makes the next split worth a great deal, so a greedy stopping rule discards it before its value appears.

1.1 The pruning family

Definition 11.1 (Tree, branch, pruned subtree). $T_{\max}$ is a finite binary tree: each node t has either two children t_L, t_R or none, in which case it is a **leaf** of $T_{\max}$. The **branch** $T_{\max, t}$ is the set of descendants of t , including t . A **pruned subtree rooted at t** , written $S \preceq T_{\max, t}$, is obtained from $T_{\max, t}$ by choosing a set of nodes and deleting everything strictly below each of them; $\mathcal{S}(t)$ denotes the collection of these, a finite set containing both $\{t\}$ and $T_{\max, t}$. Write $|S|$ for the number of leaves of S .

Definition 11.2 (Node cost). A **node cost** assigns $R(t) \geq 0$ to every node, the cost incurred if t is made a leaf, and a tree is charged the sum over its leaves,

$$R(S) = \sum_{t \text{ leaf of } S} R(t). \quad (1)$$

It satisfies the **refinement property** if $R(t) \geq R(t_L) + R(t_R)$ at every internal node.

Proposition 11.3 (The three costs of this unit refine). Let node t hold n_t observations. Each of

- (i) the residual sum of squares $R(t) = \sum_{i \in t} (y_i - \bar{y}_t)^2$,
- (ii) the misclassification count $R(t) = n_t - \max_k n_{tk}$, where n_{tk} is the number of class k observations in t ,
- (iii) $R(t) = n_t \phi(\mathbf{p}_t)$ for any concave ϕ on the probability simplex, $\mathbf{p}_t$ being the vector of class proportions in t ,

satisfies the refinement property, and (ii) is the case $\phi(\mathbf{p}) = 1 - \max_k p_k$ of (iii).

Proof. (i) is the analysis of variance identity of Unit 10, whose third term $n_L n_R (\bar{y}_L - \bar{y}_R)^2 / n_t$ is non-negative.

(iii) The class counts add, $n_{tk} = n_{Lk} + n_{Rk}$, so $\mathbf{p}_t = (n_L/n_t)\mathbf{p}_L + (n_R/n_t)\mathbf{p}_R$, and concavity gives $\phi(\mathbf{p}_t) \geq (n_L/n_t)\phi(\mathbf{p}_L) + (n_R/n_t)\phi(\mathbf{p}_R)$; multiplying by n_t gives $R(t) \geq R(t_L) + R(t_R)$.

(ii) $\mathbf{p} \mapsto \max_k p_k$ is a maximum of linear functions, hence convex, so $1 - \max_k p_k$ is concave and (iii) applies. Directly: $\max_k (n_{Lk} + n_{Rk}) \leq \max_k n_{Lk} + \max_k n_{Rk}$. $\square$

Definition 11.4 (Cost complexity). For $\alpha \geq 0$ and $S \in \mathcal{S}(t)$,

$$R_\alpha(S) = R(S) + \alpha |S|. \quad (2)$$

Remark. The number of pruned subtrees grows faster than any polynomial in the size of $T_{\max}$: a complete binary tree of depth d has N_d pruned subtrees with $N_0 = 1$ and $N_d = 1 + N_{d-1}^2$, since such a subtree is either the root alone or a pair of pruned subtrees of the two branches. That gives $N_5 = 458,330$ and $N_6 > 2 \times 10^{11}$ on a tree of 127 nodes. Minimising (2) by enumeration is not available.

1.2 The minimising subtree

Definition 11.5 (Value function). $m_\alpha(t) = \min_{S \in \mathcal{S}(t)} R_\alpha(S)$.

Lemma 11.6 (Recursion). If t is a leaf of $T_{\max}$ then $m_\alpha(t) = R(t) + \alpha$. Otherwise

$$m_\alpha(t) = \min \{ R(t) + \alpha, \quad m_\alpha(t_L) + m_\alpha(t_R) \}. \quad (3)$$

Proof. A pruned subtree rooted at an internal t either is $\{t\}$, of cost complexity $R(t) + \alpha$, or retains both children, in which case it is determined by a pair $S_L \in \mathcal{S}(t_L)$, $S_R \in \mathcal{S}(t_R)$ and its leaves are those of S_L together with those of S_R . Both R and $|\cdot|$ add over that partition of the leaves, so $R_\alpha(S) = R_\alpha(S_L) + R_\alpha(S_R)$, and minimising over the pair minimises the two terms separately. $\square$

Lemma 11.7 (Shape in α). $\alpha \mapsto m_\alpha(t)$ is the minimum of the finitely many affine functions $\alpha \mapsto R(S) + \alpha|S|$, $S \in \mathcal{S}(t)$. It is therefore continuous, piecewise linear, concave, and strictly increasing, every slope $|S|$ being at least 1.

Proof. Definition 11.5 is exactly that minimum, over a finite set. A minimum of finitely many affine functions is concave and piecewise linear, and it is strictly increasing because each of them is. $\square$

Theorem 11.8 (Every internal node has a pruning threshold). Assume the refinement property and let t be internal in $T_{\max}$. Put

$$h_t(\alpha) = m_\alpha(t_L) + m_\alpha(t_R) - R(t) - \alpha. \quad (4)$$

Then h_t is continuous, piecewise linear and strictly increasing with every slope at least 1, $h_t(0) \leq 0$, and $h_t(\alpha) \rightarrow \infty$. Hence there is a unique $\alpha_t \in [0, \infty)$ with

$$h_t(\alpha) < 0 \text{ for } \alpha < \alpha_t, \quad h_t(\alpha) \geq 0 \text{ for } \alpha \geq \alpha_t. \quad (5)$$

Proof. By Lemma 11.7 each of $m_\alpha(t_L)$ and $m_\alpha(t_R)$ is piecewise linear with slopes at least 1, so their sum has slopes at least 2 and h_t has slopes at least $2 - 1 = 1$: it is continuous, piecewise linear, strictly increasing, and unbounded above.

At $\alpha = 0$ the minimum in Definition 11.5 is over $R(S)$ alone, and iterating the refinement property down the branch shows $R(T_{\max,t}) \leq R(S)$ for every $S \in \mathcal{S}(t)$, so $m_0(t) = R(T_{\max,t})$. Hence $h_t(0) = R(T_{\max,t_L}) + R(T_{\max,t_R}) - R(t) = R(T_{\max,t}) - R(t) \leq 0$, again by refinement. A continuous strictly increasing function that is non-positive at 0 and unbounded above has a unique zero $\alpha_t \geq 0$, and (5) follows. $\square$

Definition 11.9 (The canonical subtree). $S_\alpha(t)$ is defined upward from the leaves: $S_\alpha(t) = \{t\}$ if t is a leaf of $T_{\max}$ or if $\alpha \geq \alpha_t$; otherwise $S_\alpha(t)$ is t together with $S_\alpha(t_L)$ and $S_\alpha(t_R)$. Write $T(\alpha) = S_\alpha(r)$ at the root r .

Theorem 11.10 ($T(\alpha)$ is the smallest minimiser). For every $\alpha \geq 0$ and every node t ,

$$R_\alpha(S_\alpha(t)) = m_\alpha(t), \quad \text{and} \quad S_\alpha(t) \preceq S \quad \text{for every minimiser } S \in \mathcal{S}(t). \quad (6)$$

In particular $T(\alpha)$ minimises R_α over $\mathcal{S}(r)$ and is a pruned subtree of every other minimiser.

Proof. Induction on the height of t . If t is a leaf of $T_{\max}$ then $\mathcal{S}(t) = \{\{t\}\}$ and both statements are trivial.

Let t be internal. If $\alpha \geq \alpha_t$ then $h_t(\alpha) \geq 0$, that is $R(t) + \alpha \leq m_\alpha(t_L) + m_\alpha(t_R)$, so (3) gives $m_\alpha(t) = R(t) + \alpha = R_\alpha(\{t\})$; and $S_\alpha(t) = \{t\}$ is a pruned subtree of every member of $\mathcal{S}(t)$.

If $\alpha < \alpha_t$ then $h_t(\alpha) < 0$, so $m_\alpha(t) = m_\alpha(t_L) + m_\alpha(t_R)$ and the inequality $R(t) + \alpha > m_\alpha(t)$ is strict: no minimiser is $\{t\}$. Every minimiser S therefore retains both children and splits as a pair (S_L, S_R) with $R_\alpha(S_L) + R_\alpha(S_R) = m_\alpha(t_L) + m_\alpha(t_R)$, which forces S_L and S_R to be minimisers at t_L and t_R . By the induction hypothesis $R_\alpha(S_\alpha(t_L)) + R_\alpha(S_\alpha(t_R)) = m_\alpha(t)$, which is the first claim, and $S_\alpha(t_L) \preceq S_L$, $S_\alpha(t_R) \preceq S_R$, which is the second. $\square$

Theorem 11.11 (Nesting). If $0 \leq \alpha < \alpha'$ then $T(\alpha') \preceq T(\alpha)$.

Proof. Induction on the height of t , proving $S_{\alpha'}(t) \preceq S_\alpha(t)$. For a leaf of $T_{\max}$ both are $\{t\}$. For internal t , if $\alpha' \geq \alpha_t$ then $S_{\alpha'}(t) = \{t\}$, which is a pruned subtree of anything. Otherwise $\alpha < \alpha' < \alpha_t$, so both $S_{\alpha'}(t)$ and $S_\alpha(t)$ retain both children, and the induction hypothesis applies at t_L and t_R . $\square$

Corollary 11.12 (A finite nested sequence). There are α values $0 = \eta_0 < \eta_1 < \cdots < \eta_M$ and trees $T_{\max} = T^{(0)} \succ T^{(1)} \succ \cdots \succ T^{(M)} = \{r\}$ with $T(\alpha) = T^{(j)}$ for $\alpha \in [\eta_j, \eta_{j+1})$ and $T(\alpha) = T^{(M)}$ for $\alpha \geq \eta_M$. Moreover $M < |T_{\max}|$, so cross-validation over α is a search over fewer than $|T_{\max}|$ candidate trees.

Proof. By Definition 11.9, $T(\alpha)$ depends on α only through which of the finitely many inequalities $\alpha \geq \alpha_t$ hold, so it is piecewise constant with breakpoints among $\{\alpha_t\}$. By Theorem 11.11 the distinct values are strictly nested, and each strict pruning removes at least one leaf, so there are at most $|T_{\max}|$ of them. At $\alpha = 0$ the value is $T_{\max}$, since $h_t(0) \leq 0$ makes $\alpha_t \geq 0$ and the definition expands wherever $\alpha < \alpha_t$; for α past every α_t it is $\{r\}$. $\square$

Corollary 11.13 (Weakest link). Suppose at $\alpha = \alpha_t$ the branch below t is not itself pruned, that is $S_{\alpha_t}(t_L)$ and $S_{\alpha_t}(t_R)$ together form a tree T_t with $|T_t| > 1$ leaves. Then

$$\alpha_t = \frac{R(t) - R(T_t)}{|T_t| - 1}. \quad (7)$$

Proof. $h_t(\alpha_t) = 0$ reads $m_{\alpha_t}(t_L) + m_{\alpha_t}(t_R) = R(t) + \alpha_t$, and by Theorem 11.10 the left side is $R_{\alpha_t}(T_t) = R(T_t) + \alpha_t|T_t|$. Solve for α_t . $\square$

Remark. Equation (7) is the quantity every implementation reports: the cost saved by keeping the branch, per extra leaf. **tree** prints it as the vector **k** of a pruning sequence; **rpart** prints **cp**, which is (7) divided by the cost of the root, so that **cp** is a fraction of the total.

Remark. The proof is an algorithm. One sweep from the leaves upward evaluates m_α at every node by (3), so the whole nested sequence costs a number of operations proportional to the number of nodes times the number of breakpoints, against the super-exponential count of pruned subtrees.

1.3 Choosing the penalty

Definition 11.14 (Cross-validation over the sequence). Split the data into K folds. For each fold k , grow a tree on the other $K - 1$ folds, form its own pruning sequence, and record the cost on fold k of the subtree selected at each α in a common grid. Average over folds to obtain $CV(\alpha)$ and take $\hat{\alpha} = \arg \min_{\alpha} CV(\alpha)$. The tree reported is $T(\hat{\alpha})$ from the sequence grown on all the data.

Remark. The grid must be common to all folds, and the fold sequences have their own breakpoints, so the grid is taken from the full-data sequence. Two consequences are easy to overlook. The subtree a fold contributes at a given α need not have the number of leaves that the full-data tree has at that α , so the size axis of a cross-validation plot is a label from the full data and not a property of what was scored; and the whole tree, not only its pruning, must be regrown inside each fold, since choosing the splits on all the data and cross-validating only the pruning would evaluate a rule that has already seen the held-out observations.

Definition 11.15 (One standard error rule). Let $\widehat{\text{se}}(\alpha)$ be the standard error over folds of the cross-validated cost. The **one standard error rule** selects the largest α , that is the smallest tree, with

$$\text{CV}(\alpha) \leq \text{CV}(\hat{\alpha}) + \widehat{\text{se}}(\hat{\alpha}). \quad (8)$$

Remark. Corollary 11.12 makes Definition 11.14 feasible: the candidate set is a nested sequence of fewer than $|T_{\max}|$ trees, not the collection of all pruned subtrees. Nothing here shows that $\hat{\alpha}$ estimates the minimiser of the test error well; that is the cross-validation question of Unit 1.

2 Classification Trees

ISLR §8.1.2.

2.1 Leaf labels

Definition 11.16 (Classification tree). The response takes values in $\{1, \dots, K\}$. A classification tree assigns to each leaf t the class proportions $\mathbf{p}_t = (p_{t1}, \dots, p_{tK})$ with $p_{tk} = n_{tk}/n_t$, and predicts a single class $\hat{k}(t)$ for every observation falling in t .

Proposition 11.17 (Majority vote). The label minimising the number of training errors in leaf t is any modal class $\hat{k}(t) \in \arg \max_k n_{tk}$, and the resulting error count is $n_t - \max_k n_{tk} = n_t M(\mathbf{p}_t)$ with $M(\mathbf{p}) = 1 - \max_k p_k$.

Proof. Labelling t with class k misclassifies the $n_t - n_{tk}$ observations of other classes, which is smallest when n_{tk} is largest. $\square$

Remark. Unit 1 proved that the classifier minimising the error probability assigns $\mathbf{x}$ to $\arg \max_k \mathbf{P}\{Y = k \mid X = \mathbf{x}\}$. Proposition 11.17 is that rule with the conditional probabilities replaced by the proportions in the leaf, so a classification tree is an estimate of the Bayes classifier in which the conditioning event is the box rather than the point. The class proportions themselves, not only the label, are part of the output: two leaves may share a label and differ entirely in how confident it is.

2.2 Impurity

Definition 11.18 (Impurity function). An **impurity** is a continuous concave $\phi : \Delta^{K-1} \rightarrow [0, \infty)$ on the probability simplex $\Delta^{K-1} = \{\mathbf{p} \geq \mathbf{0} : \sum_k p_k = 1\}$ with $\phi(\mathbf{e}_k) = 0$ at every vertex. The impurity of a tree is $R(T) = \sum_{t \text{ leaf}} n_t \phi(\mathbf{p}_t)$ and the **gain** of a split is

$$\Delta_\phi = \phi(\mathbf{p}_t) - \frac{n_L}{n_t} \phi(\mathbf{p}_L) - \frac{n_R}{n_t} \phi(\mathbf{p}_R). \quad (9)$$

Definition 11.19 (The three impurities).

$$M(\mathbf{p}) = 1 - \max_k p_k, \quad G(\mathbf{p}) = \sum_k p_k(1 - p_k) = 1 - \sum_k p_k^2, \quad H(\mathbf{p}) = - \sum_k p_k \ln p_k, \quad (10)$$

the **misclassification rate**, the **Gini index** and the **entropy**, with $0 \ln 0 = 0$.

Proposition 11.20 (Concavity). M , G and H are impurities. G and H are strictly concave on the simplex; M is concave but affine on every region where a single class is strictly modal.

Proof. All three vanish at the vertices and are continuous, $x \ln x \rightarrow 0$ as $x \downarrow 0$ giving continuity of H .

G : $\mathbf{p} \mapsto - \sum_k p_k^2$ has Hessian $-2\mathbf{I}$, negative definite, so G is strictly concave.

H : $x \mapsto -x \ln x$ has second derivative $-1/x < 0$ on $(0, 1]$, so the Hessian of H is diagonal with strictly negative entries on the interior, and concavity extends to the boundary by continuity.

M : $\max_k p_k$ is a maximum of linear functions, hence convex, so M is concave. On the set where class k^* is strictly modal, $M(\mathbf{p}) = 1 - p_{k^*}$ is affine. $\square$

Theorem 11.21 (A split never hurts, and when it does nothing). For any impurity ϕ , $\Delta_\phi \geq 0$. If ϕ is strictly concave and both children are non-empty, $\Delta_\phi = 0$ if and only if $\mathbf{p}_L = \mathbf{p}_R$.

Proof. The class counts add, so $\mathbf{p}_t = (n_L/n_t)\mathbf{p}_L + (n_R/n_t)\mathbf{p}_R$ and concavity gives $\phi(\mathbf{p}_t) \geq (n_L/n_t)\phi(\mathbf{p}_L) + (n_R/n_t)\phi(\mathbf{p}_R)$, which is (9) ≥ 0 . Strict concavity makes the inequality strict unless $\mathbf{p}_L = \mathbf{p}_R$, in which case both sides equal $\phi(\mathbf{p}_t)$. $\square$

Theorem 11.22 (Misclassification is blind to a split that keeps the modal class). If the same class k^* is modal in the parent, in the left child and in the right child, then $\Delta_M = 0$ exactly.

Proof. Under the hypothesis, $n_t M(\mathbf{p}_t) = n_t - n_{tk^*}$ and likewise for the children, and the counts add: $n_L M(\mathbf{p}_L) + n_R M(\mathbf{p}_R) = (n_L - n_{Lk^*}) + (n_R - n_{Rk^*}) = n_t - n_{tk^*} = n_t M(\mathbf{p}_t)$. Divide by n_t . $\square$

Remark. Trees are therefore not grown on the error count. A split that leaves both children with the same modal class scores exactly zero on M however differently the two children are composed, so a greedy search on M has no gradient to follow and stops. Under G or H the same split has

strictly positive gain unless the two child distributions coincide (Theorem 11.21), and the search continues.

Theorem 11.23 (Ordering). For every $\mathbf{p} \in \Delta^{K-1}$,

$$M(\mathbf{p}) \leq G(\mathbf{p}) \leq H(\mathbf{p}), \quad (11)$$

and for $K = 2$ also $G(\mathbf{p}) \leq 2M(\mathbf{p})$, with equality at $\mathbf{p} = (\frac{1}{2}, \frac{1}{2})$.

Proof. For the first inequality, $\sum_k p_k^2 \leq (\max_k p_k) \sum_k p_k = \max_k p_k$, so $G = 1 - \sum_k p_k^2 \geq 1 - \max_k p_k = M$.

For the second, $\ln u \leq u - 1$ for $u > 0$, applied at $u = p_k$, gives $-\ln p_k \geq 1 - p_k$ and hence $H = \sum_k p_k(-\ln p_k) \geq \sum_k p_k(1 - p_k) = G$, the terms with $p_k = 0$ contributing 0 to both sides.

For $K = 2$ write $\mathbf{p} = (p, 1 - p)$ and $q = \min\{p, 1 - p\}$, so $M = q$ and $G = 2p(1 - p) = 2q(1 - q) \leq 2q$ since $1 - q \leq 1$. Equality needs $q = 0$, where both sides vanish, or $q = \frac{1}{2}$. $\square$

Proposition 11.24 (What the Gini index measures). Let U, V be independent draws from $\mathbf{p}$. Then $G(\mathbf{p}) = \mathbf{P}\{U \neq V\}$: it is the error rate of the rule that guesses a class at random from the leaf's own proportions. For $K = 2$, $G(\mathbf{p}) = 2 \operatorname{var}(\mathbf{1}\{U = 1\})$.

Proof. $\mathbf{P}\{U = V\} = \sum_k p_k^2$. For $K = 2$, $\operatorname{var}(\mathbf{1}\{U = 1\}) = p(1 - p)$ and $G = 2p(1 - p)$. $\square$

Proposition 11.25 (Entropy and the reported deviance). The quantity $-2 \sum_{t \text{ leaf}} \sum_k n_{tk} \ln p_{tk}$, which `tree` prints as the deviance of a classification tree, equals $2 \sum_{t \text{ leaf}} n_t H(\mathbf{p}_t)$. It is the log likelihood ratio statistic of Unit 7 comparing the fitted leaf probabilities with a saturated model.

Proof. $\sum_k n_{tk} \ln p_{tk} = n_t \sum_k p_{tk} \ln p_{tk} = -n_t H(\mathbf{p}_t)$. For the second sentence, the multinomial log likelihood of leaf t is $\sum_k n_{tk} \ln p_{tk}$ up to a constant free of the parameters, and the saturated model puts each observation's own class at probability one, with log likelihood 0. $\square$

Remark. The three quantities therefore serve different roles. G or H grows the tree, because Theorem 11.22 says M cannot. M prunes it, because Corollary 11.12 needs a cost and the cost one actually cares about is the error count. Both are legitimate node costs by Proposition 11.3,

and the two choices can select trees differing by twenty leaves on the same data.

2.3 Two classes

Theorem 11.26 (A two-class Gini split is a regression split on the indicator). Let $K = 2$ and code $y_i = \mathbf{1}\{\text{observation } i \text{ is in class 2}\}$. Then for any split of node t ,

$$n_L G(\mathbf{p}_L) + n_R G(\mathbf{p}_R) = 2 \left\{ \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2 \right\}, \quad (12)$$

so minimising the weighted Gini impurity and minimising the residual sum of squares of the 0–1 response select the same split, and the Gini gain is $2n_L n_R (\bar{y}_L - \bar{y}_R)^2 / n_t$.

Proof. In a node holding m observations with $\bar{y}$ of them coded 1 in proportion, $\sum_i (y_i - \bar{y})^2 = m\bar{y}(1 - \bar{y})$ because $y_i^2 = y_i$. The class proportions are $\mathbf{p} = (1 - \bar{y}, \bar{y})$, so $G(\mathbf{p}) = 2\bar{y}(1 - \bar{y})$ and $mG(\mathbf{p}) = 2 \sum_i (y_i - \bar{y})^2$. Applying this in each child gives (12). The gain expression is the analysis of variance identity of Unit 10 applied to the 0–1 response, multiplied by 2. $\square$

Remark. Nothing new has to be implemented for two-class Gini splitting: it is the regression splitter of Unit 10 run on the indicator. The identity also explains why the Gini gain carries the same $n_L n_R / n_t$ weight, so a split peeling off a handful of observations is penalised for the same reason.

2.4 Categorical predictors

A predictor with L unordered levels admits $2^{L-1} - 1$ non-trivial splits, which is already $2^{31} - 1$ at $L = 32$. The search collapses to $L - 1$ candidates.

Theorem 11.27 (Ordered search suffices). Let x be categorical with levels $1, \dots, L$, let $\bar{y}_l$ be the mean response of the observations at level l , and suppose the $\bar{y}_l$ are distinct and ordered $\bar{y}_1 < \dots < \bar{y}_L$. Then some subset A^* minimising

$$Q(A) = \sum_{i: x_i \in A} (y_i - \bar{y}_A)^2 + \sum_{i: x_i \notin A} (y_i - \bar{y}_{A^c})^2 \quad (13)$$

over non-trivial A is an initial segment $\{1, \dots, m\}$. By Theorem 11.26 the same holds for a two-class Gini split, with $\bar{y}_l$ the proportion of class 2 at level l .

Proof. Write $Q(A) = \min_{a,b} F(A, a, b)$ with $F(A, a, b) = \sum_{x_i \in A} (y_i - a)^2 + \sum_{x_i \notin A} (y_i - b)^2$, the minimum over a and b being attained at the two group means. So a minimiser of Q is part of a minimiser of F over all three arguments.

Fix a and b and minimise F over A . All observations at a level move together, and level l contributes $\sum_{i \in l} (y_i - a)^2$ if $l \in A$ and $\sum_{i \in l} (y_i - b)^2$ otherwise. Splitting each at $\bar{y}_l$,

$$\sum_{i \in l} (y_i - a)^2 - \sum_{i \in l} (y_i - b)^2 = n_l \{ (\bar{y}_l - a)^2 - (\bar{y}_l - b)^2 \} = n_l (b - a) (2\bar{y}_l - a - b),$$

the within-level sums of squares cancelling. Hence, when $a < b$, the contribution is smaller by putting l in A exactly when $\bar{y}_l < (a + b)/2$, and larger exactly when $\bar{y}_l > (a + b)/2$.

Let A^* minimise Q and put $a = \bar{y}_{A^*}$, $b = \bar{y}_{A^{*c}}$. If $a = b$ the analysis of variance identity makes $Q(A^*)$ equal to the total sum of squares in the node, so no split reduces it and every A , the initial segments included, is a minimiser. Otherwise assume $a < b$, exchanging the names of A^* and A^{*c} if necessary. Since (A^*, a, b) minimises F , no level can be moved to reduce it, so every $l \in A^*$ has $\bar{y}_l \leq (a + b)/2$ and every $l \notin A^*$ has $\bar{y}_l \geq (a + b)/2$; distinctness rules out both at once. Hence A^* is exactly the set of levels with $\bar{y}_l$ below the midpoint, an initial segment of the ordering. $\square$

Remark. The proof is the two-mean assignment step: put each level with whichever of the two fitted values it is closer to. Sorting the levels once and scanning the $L - 1$ cutpoints therefore finds an optimal split at the cost of Unit 10's continuous case. Theorem 11.27 needs a scalar $\bar{y}_l$ to sort by, so it covers regression and two-class classification; with $K > 2$ classes there is no such ordering and implementations fall back on heuristics.

3 Trees Against Linear Models

ISLR §8.1.3–8.1.4.

A linear model fits $f(\mathbf{x}) = \beta_0 + \sum_j \beta_j x_j$ and a tree fits $f(\mathbf{x}) = \sum_m c_m \mathbf{1}\{\mathbf{x} \in R_m\}$. Neither contains the other.

3.1 The cost of a staircase

Theorem 11.28 (Approximating a linear function by boxes). Let X be uniform on $[0, 1]$ and $f(x) = \beta_0 + \beta_1 x$. Among all partitions of $[0, 1]$ into J intervals, the mean squared error of the

best piecewise-constant approximation,

$$\min_{\text{partitions}} \mathbb{E}[\{f(X) - \mathbb{E}[f(X) \mid X \in I(X)]\}^2] = \frac{\beta_1^2}{12J^2}, \quad (14)$$

is attained by the J intervals of equal width. In p dimensions, with X uniform on $[0, 1]^p$, $f(\mathbf{x}) = \beta_0 + \boldsymbol{\beta}'\mathbf{x}$ and a grid of $J = m^p$ equal boxes,

$$\mathbb{E}[\{f(X) - \mathbb{E}[f(X) \mid \text{box}]\}^2] = \frac{|\boldsymbol{\beta}|^2}{12 J^{2/p}}. \quad (15)$$

Proof. On an interval I of width w the conditional law of X is uniform, so $\text{var}(f(X) \mid X \in I) = \beta_1^2 w^2 / 12$, and the error contributed by I is $\mathbb{P}\{X \in I\} \text{var}(f(X) \mid X \in I) = \beta_1^2 w^3 / 12$. Summing, the error is $(\beta_1^2 / 12) \sum_j w_j^3$ subject to $\sum_j w_j = 1$. The function $u \mapsto u^3$ is convex, so Jensen's inequality (prerequisites) gives $J^{-1} \sum_j w_j^3 \geq (J^{-1} \sum_j w_j)^3 = J^{-3}$, that is $\sum_j w_j^3 \geq J^{-2}$, with equality when all w_j are equal. That is (14).

In p dimensions the coordinates of X are independent, and on a box of side $1/m$ in every coordinate

the conditional variance of $\beta'X$ is $\sum_j \beta_j^2/(12m^2)$, free of which box. With $J = m^p$, $m^2 = J^{2/p}$. $\square$

Remark. Halving the error of a linear truth costs $\sqrt{2}$ times as many leaves in one dimension and $2^{p/2}$ times as many in p : at $p = 10$ a factor of 32. A linear model has no such error at all. The difficulty is dimensional, not a matter of sample size.

3.2 Where a tree is exactly right

Theorem 11.29 (A single step). Let X be uniform on $[0, 1]$ and $f(x) = \mathbf{1}\{x > c\}$ with $c \in (0, 1)$. The best linear predictor of $f(X)$ has mean squared error

$$c(1 - c)\{1 - 3c(1 - c)\}, \quad (16)$$

which is $1/16$ at $c = \frac{1}{2}$, while the two-leaf tree that splits at c has mean squared error 0. Moreover the split gain of Unit 10, in population form $s \mapsto \mathbf{P}\{X \leq s\} \mathbf{P}\{X > s\} \{\mathbf{E}[f \mid X \leq s] - \mathbf{E}[f \mid X > s]\}^2$, is maximised at $s = c$, so greedy splitting finds the right cut.

Proof. $E[f] = 1 - c$ and $f^2 = f$, so $\text{var}(f) = c(1 - c)$; $\text{var}(X) = 1/12$ and $\text{cov}(f, X) = \int_c^1 x \, dx - (1 - c)/2 = (1 - c^2)/2 - (1 - c)/2 = c(1 - c)/2$. The squared correlation is therefore $\{c(1 - c)/2\}^2 / \{c(1 - c)/12\} = 3c(1 - c)$, and the best linear predictor leaves $\text{var}(f)\{1 - \text{corr}^2\}$, which is (16); at $c = \frac{1}{2}$ it is $\frac{1}{4}(1 - \frac{3}{4}) = \frac{1}{16}$. The tree that splits at c predicts 0 below and 1 above, reproducing f .

For the gain, take $s \leq c$: the left piece has $f \equiv 0$ so its mean is 0, and the right piece has mean $(1 - c)/(1 - s)$, giving $s(1 - s)\{(1 - c)/(1 - s)\}^2 = s(1 - c)^2/(1 - s)$, increasing in s on $[0, c]$. For $s \geq c$ the right piece has $f \equiv 1$ and the left has mean $(s - c)/s$, giving $s(1 - s)\{1 - (s - c)/s\}^2 = (1 - s)c^2/s$, decreasing in s . Both expressions equal $c(1 - c)$ at $s = c$. $\square$

Theorem 11.30 (A structure a tree can hold but greedy growing cannot find). Let $X = (X_1, X_2)$ be uniform on $[0, 1]^2$ and $f(\mathbf{x}) = \mathbf{1}\{x_1 > \frac{1}{2}\}$ exclusive-or $\mathbf{1}\{x_2 > \frac{1}{2}\}$, that is $f = 1$ on two opposite quarters and 0 on the other two. Then

- (i) a tree with four leaves reproduces f exactly;
- (ii) the best linear predictor of $f(X)$ is the constant $\frac{1}{2}$, with mean squared error $\frac{1}{4}$;
- (iii) every single axis-aligned split of $[0, 1]^2$ has population split gain exactly 0.

Proof. (i) Split at $x_1 = \frac{1}{2}$ and then each half at $x_2 = \frac{1}{2}$.

(ii) $\mathbf{E}[f] = \frac{1}{2}$. For the covariance with X_1 ,

$$\mathbf{E}[fX_1] = \int_{1/2}^1 \int_{1/2}^1 x_1 + \int_0^{1/2} \int_0^{1/2} x_1 = \frac{1}{2} \int_{1/2}^1 x \, dx + \frac{1}{2} \int_0^{1/2} x \, dx = \frac{1}{2} \cdot \frac{3}{8} + \frac{1}{2} \cdot \frac{1}{8} = \frac{1}{4},$$

which equals $\mathbf{E}[f] \mathbf{E}[X_1]$, so $\text{cov}(f, X_1) = 0$, and by symmetry $\text{cov}(f, X_2) = 0$. The best linear predictor is therefore the constant $\mathbf{E}[f] = \frac{1}{2}$ and its error is $\text{var}(f) = \frac{1}{4}$.

(iii) Take a split at $X_1 = s$ with $s \leq \frac{1}{2}$. On the left, $x_1 \leq s \leq \frac{1}{2}$, so $f = \mathbf{1}\{x_2 \leq \frac{1}{2}\}$ and the mean is $\frac{1}{2}$. On the right, conditionally on X_1 the mean of f is $\frac{1}{2}$ whichever side of $\frac{1}{2}$ X_1 falls, so the mean is $\frac{1}{2}$ there too. The two conditional means coincide and the gain, which by Unit 10 is a positive multiple of their squared difference, vanishes. The cases $s > \frac{1}{2}$ and splits on X_2 are the same computation. $\square$

Remark. Part (iii) is the limitation that matters and it is not the one usually stated. The target is a partition predictor with four boxes, exactly the class a tree fits, and the tree still fails: the greedy criterion of Unit 10 sees nothing at the first split, so whatever the algorithm does next

is driven by sampling noise. What a tree can represent and what growing it finds are different questions.

3.3 Instability

Remark. A tree is a discontinuous function of the data. The split chosen at a node is the $\arg \max$ of finitely many gains, so an arbitrarily small perturbation that reverses the order of the top two changes the split, and every box below it. Nothing above bounds the resulting change in the fitted function. On **Boston**, two trees grown on disjoint halves agree on the root variable about a fifth of the time, and their predictions differ by roughly 45% of the response's own standard deviation.

Remark. Instability is a variance, and Unit 1's decomposition says variance can be reduced by averaging.

4 Computation: Formulas Against Package Output

ISLR §8.3.1–8.3.2.

Boston from **ISLR2** for regression, continuing from Unit 10, and **Carseats** from the same package for classification, with the binary response $\text{High} = \mathbf{1}\{\text{Sales} > 8\}$ that the ISLR lab uses.

```

1 suppressMessages({library(ISLR2); library(tree); library(rpart)})
2 Car <- Carseats
3 Car$High <- factor(ifelse(Car$Sales <= 8, "No", "Yes"))
4 tb <- tree(medv ~ ., Boston)
5 tc <- tree(High ~ . - Sales, Car)
6 c(n_boston = nrow(Boston), leaves_boston = sum(tb$frame$var == "<leaf>"),
7   n_carseats = nrow(Car), leaves_carseats = sum(tc$frame$var == "<leaf>"))

```

```

##           n_boston  leaves_boston      n_carseats
##           506          9             400
## leaves_carseats
##           27

```

4.1 The pruning sequence (§1.2)

```

1 fr <- tb$frame; id <- as.integer(rownames(fr))
2 Rn <- setNames(fr$dev, id); lf <- setNames(fr$var == "<leaf>", id)
3 mval <- function(a, t) { k <- as.character(t)
4   if (lf[[k]]) Rn[[k]] + a else
5     min(Rn[[k]] + a, mval(a, 2*t) + mval(a, 2*t + 1)) }
6 alpha_t <- function(t) uniroot(function(a)

```

```

7   mval(a, 2*t) + mval(a, 2*t + 1) - Rn[[as.character(t)]] - a,
8   c(0, 1e7), tol = 1e-9)$root
9 manual <- sort(sapply(id[!lf], alpha_t))
10 pk <- prune.tree(tb)$k[-1]
11 signif(rbind(manual_thresholds = manual, package_k = pk,
12              difference = manual - pk), 5)

```

```

##           [,1]      [,2]      [,3]      [,4]
## manual_thresholds 4.455e+02 7.256e+02 1.137e+03 1.442e+03
## package_k        4.455e+02 7.256e+02 1.137e+03 1.442e+03
## difference        -1.137e-13 -3.411e-13 2.274e-13 -4.547e-13
##           [,5]      [,6]      [,7]      [,8]
## manual_thresholds 2.520e+03 3.061e+03 7.312e+03 19340
## package_k        2.520e+03 3.061e+03 7.312e+03 19340
## difference        9.095e-13 -9.095e-13 1.091e-11      0

```

```

1 leafset <- function(a, t) { k <- as.character(t)
2   if (lf[[k]] || Rn[[k]] + a <= mval(a, 2*t) + mval(a, 2*t + 1)) t
3   else c(leafset(a, 2*t), leafset(a, 2*t + 1)) }
4 grid <- c(0, (head(manual, -1) + tail(manual, -1))/2, max(manual) + 1)
5 t(sapply(grid, function(a)

```

```

6  c(alpha = a, manual_leaves = length(leafset(a, 1)),
7    package_leaves = sum(prune.tree(tb, k = a)$frame$var == "<leaf>"))))

```

```

##          alpha manual_leaves package_leaves
## [1,]      0.0           9           9
## [2,]    585.5           8           8
## [3,]    931.2           7           7
## [4,]   1289.4           6           6
## [5,]   1981.1           5           5
## [6,]   2790.6           4           4
## [7,]   5186.4           3           3
## [8,]  13325.7           2           2
## [9,]  19340.6           1           1

```

`rpart` reports the same thresholds divided by the cost of the root (Corollary 11.13).

```

1  rp <- rpart(medv ~ ., Boston, control = rpart.control(cp = 0, xval = 0))
2  R0 <- sum((Boston$medv - mean(Boston$medv))^2)
3  signif(rbind(cp = head(rp$cptable[, "CP"], 4),
4              cp_times_root_cost = head(rp$cptable[, "CP"], 4)*R0,
5              tree_thresholds = rev(manual)[1:4]), 5)

```

##	1	2	3	4
## cp	0.4527	0.1712	0.07166	0.03616
## cp_times_root_cost	19340.0000	7311.9000	3061.00000	1544.80000
## tree_thresholds	19340.0000	7311.9000	3061.00000	2520.30000

The first three agree exactly: both packages make the same first three splits, so they prune the same branches at the same prices. They part at the fourth because their stopping rules differ, **tree** using **mindev** and **minsize** where **rpart** uses **minsplit** and **minbucket**, so the trees being pruned are not the same tree.

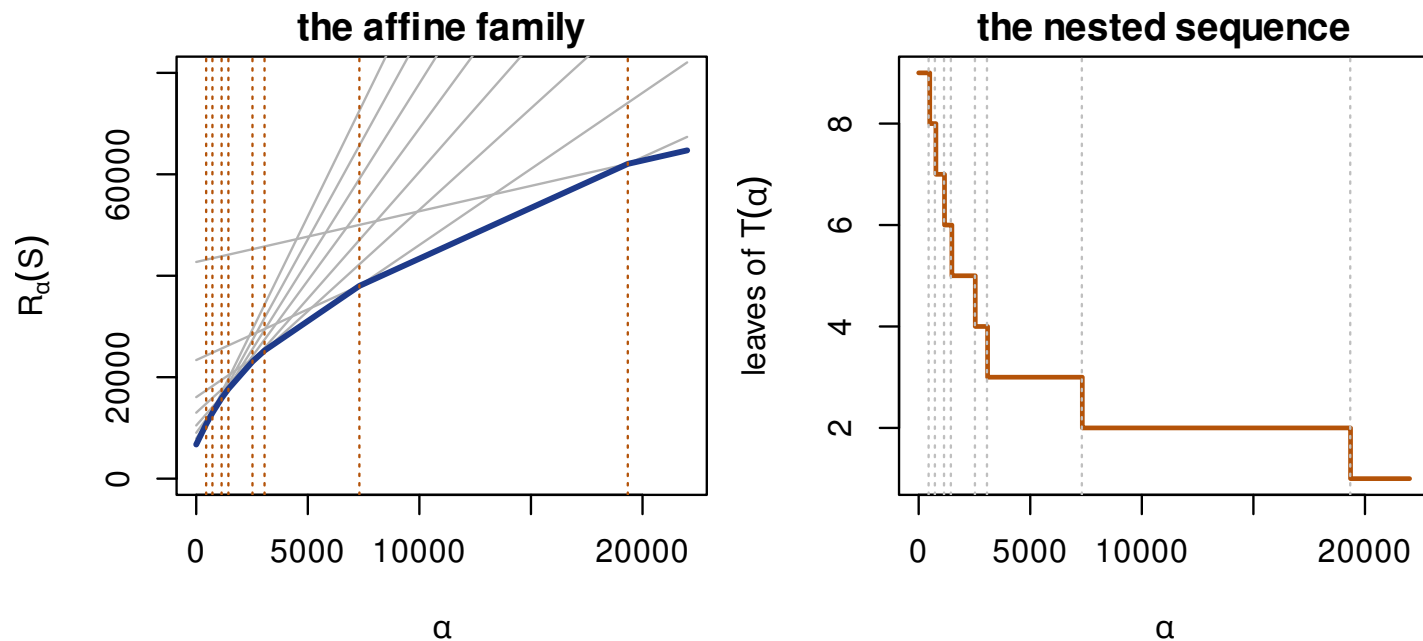


Figure 11.1: Left: for each subtree in the sequence, the line $\alpha \mapsto R(S) + \alpha|S|$, with the value function $m_\alpha(r)$ of Definition 11.5 as their lower envelope (Lemma 11.7), concave and piecewise linear; the dotted verticals are the thresholds of Theorem 11.8. Right: the leaf count of $T(\alpha)$, a decreasing step function whose jumps are exactly those thresholds (Theorem 11.11).

4.2 Cross-validation (§1.3)

`cv.tree` forms its folds by drawing a fold label for each observation independently, so the folds are not of equal size.

```
1 set.seed(3); K <- 10; n <- nrow(Boston)
2 fold <- sample(K, n, replace = TRUE)
3 c(intended_size = n/K, smallest = min(table(fold)),
4   largest = max(table(fold)))
```

```
## intended_size      smallest      largest
##              50.6           38.0           60.0
```

Each fold's tree is pruned at the **full-data** thresholds, so the subtree a fold contributes at a given α need not have the number of leaves printed against it.

```
1 mf <- model.frame(tb)
2 sz <- sapply(1:4, function(i) {
3   t1 <- tree(model = mf[fold != i, , drop = FALSE])
4   sapply(manual, function(a) sum(prune.tree(t1, k = a)$frame$var == "<leaf>")) })
5 rbind(alpha = round(manual), full_data_size = prune.tree(tb)$size[-1], t(sz))
```

```
##              [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8]
```

## alpha	445	726	1137	1442	2520	3061	7312	19340
## full_data_size	8	7	6	5	4	3	2	1
##	8	7	5	4	4	4	2	1
##	7	7	6	5	4	3	2	1
##	10	8	6	5	5	3	2	1
##	9	7	6	6	5	3	3	1

```
1 set.seed(3); cvb <- cv.tree(tb, K = 10)
2 rbind(size = cvb$size, cv_deviance = round(cvb$dev))
```

##	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]
## size	9	8	7	6	5	4	3	2
## cv_deviance	12767	13071	13350	13442	15607	17788	18590	27193
##	[,9]							
## size	1							
## cv_deviance	42844							

Cross-validation keeps the whole tree here: the deviance is smallest at nine leaves, the largest the growing rule produced. Pruning is not always an improvement, and the sequence is a family to search, not a prescription to shrink.

The one standard error rule of Definition 11.15 needs a standard error, which **rpart** supplies as **xstd**.

```

1 set.seed(5)
2 rp2 <- rpart(medv ~ ., Boston, control = rpart.control(cp = 0.001, xval = 10))
3 ct <- rp2$cptable; j <- which.min(ct[, "xerror"])
4 thr <- ct[j, "xerror"] + ct[j, "xstd"]
5 c(minimising_nsplitt = ct[j, "nsplitt"], min_xerror = ct[j, "xerror"],
6   one_se_threshold = thr,
7   one_se_nsplitt = ct[which(ct[, "xerror"] <= thr)[1], "nsplitt"])

```

```

## minimising_nsplitt      min_xerror  one_se_threshold
##           23.0000          0.2443          0.2804
##      one_se_nsplitt
##           7.0000

```

4.3 Impurity (§2.2)

```

1 Mf <- function(p) 1 - max(p); Gf <- function(p) 1 - sum(p^2)
2 Hf <- function(p) { p <- p[p > 0]; -sum(p*log(p)) }
3 set.seed(1); P <- matrix(rexp(4000*4), ncol = 4); P <- P/rowSums(P)
4 c(smallest_G_minus_M = min(apply(P, 1, Gf) - apply(P, 1, Mf)),
5   smallest_H_minus_G = min(apply(P, 1, Hf) - apply(P, 1, Gf)),
6   largest_G_minus_2M_two_classes = max(sapply(seq(0, 1, 0.001),

```

```

7  function(p) Gf(c(p, 1 - p)) - 2*Mf(c(p, 1 - p))))

##                smallest_G_minus_M                smallest_H_minus_G
##                0.01403                        0.07013
## largest_G_minus_2M_two_classes
##                0.00000

```

Two splits of a node holding 400 of each class: A into (300, 100) and (100, 300), B into (200, 400) and (200, 0).

```

1  wt <- function(nL, nR, f) { N <- sum(nL) + sum(nR)
2    sum(nL)/N*f(nL/sum(nL)) + sum(nR)/N*f(nR/sum(nR)) }
3  imp <- function(nL, nR) c(M = wt(nL, nR, Mf), G = wt(nL, nR, Gf),
4    H = wt(nL, nR, Hf))
5  round(rbind(parent = c(M = Mf(c(.5, .5)), G = Gf(c(.5, .5)),
6    H = Hf(c(.5, .5))),
7    split_A = imp(c(300, 100), c(100, 300)),
8    split_B = imp(c(200, 400), c(200, 0))), 4)

```

```

##           M           G           H
## parent  0.50 0.5000 0.6931
## split_A 0.25 0.3750 0.5623

```

```
| ## split_B 0.25 0.3333 0.4774
```

Both splits reduce M from 0.50 to 0.25, so the error count cannot choose between them; G and H both prefer B, which produces a pure leaf. Proposition 11.25 identifies what **tree** prints for a classification tree.

```
1 lc <- tc$frame$var == "<leaf>"
2 c(package_deviance = deviance(tc),
3   manual_2_sum_n_H = 2*sum(tc$frame$n[lc]*apply(tc$frame$yprob[lc, ], 1, Hf)),
4   training_errors = sum(tc$frame$n[lc]*apply(tc$frame$yprob[lc, ], 1, Mf)))
```

```
| ## package_deviance manual_2_sum_n_H training_errors
| ##                170.7             170.7          36.0
```

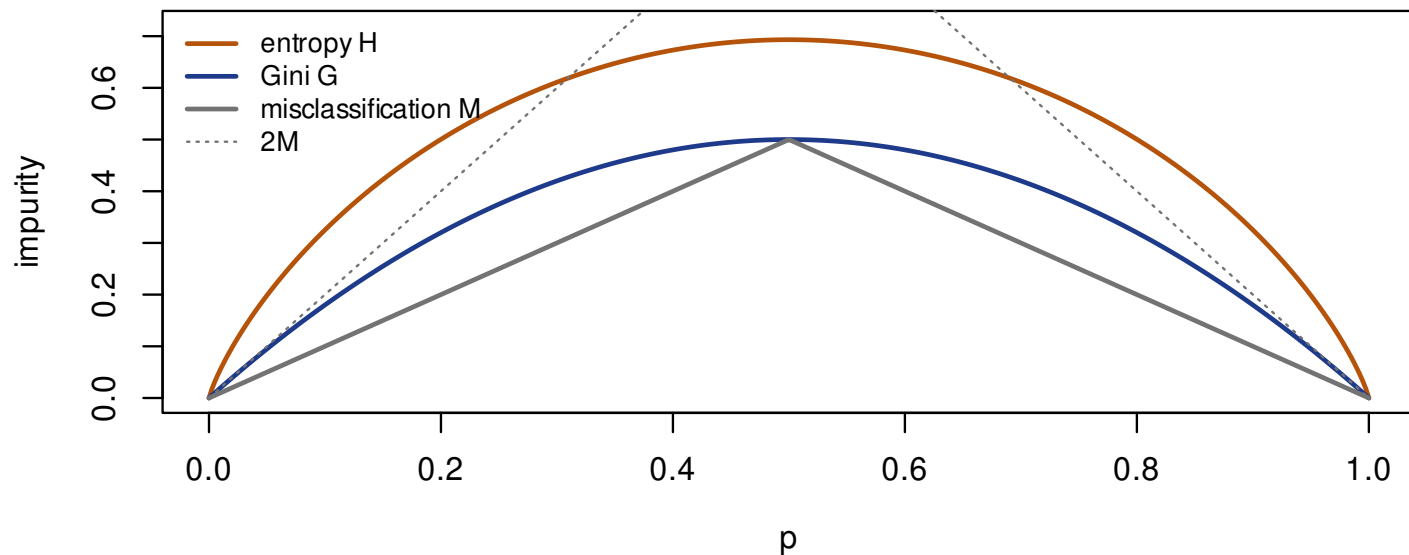


Figure 11.2: The three impurities for two classes. Theorem 11.23 places M below G below H and traps G between M and $2M$; all three vanish at $p = 0$ and $p = 1$ and peak at $p = \frac{1}{2}$. Only M has a corner there, and it is affine on each side, which is Theorem 11.22 in one picture.

4.4 Categorical predictors (§2.4)

```

1 set.seed(4); m <- 300; L <- 8
2 lv <- factor(sample(letters[1:L], m, TRUE))
3 yc <- c(2, 9, 4, 7, 1, 5, 8, 3)[as.integer(lv)] + rnorm(m, 0, 2)
4 Q <- function(A) { i <- lv %in% levels(lv)[A]
5   if (!any(i) || all(i)) Inf else
6     sum((yc[i] - mean(yc[i]))^2) + sum((yc[!i] - mean(yc[!i]))^2) }
7 subs <- lapply(1:(2^(L - 1) - 1),
8               function(b) which(as.integer(intToBits(b))[1:L] == 1))
9 ord <- order(tapply(yc, lv, mean))
10 c(subsets_tried = length(subs), ordered_tried = L - 1,
11   best_exhaustive = min(sapply(subs, Q)),
12   best_ordered = min(sapply(1:(L - 1), function(k) Q(ord[1:k]))),
13   difference = min(sapply(1:(L - 1), function(k) Q(ord[1:k])) -
14                   min(sapply(subs, Q)))

```

##	subsets_tried	ordered_tried	best_exhaustive
##	127	7	1668
##	best_ordered	difference	
##	1668	0	

4.5 Trees against linear models (§3.1–§3.2)

With $\beta_1 = 3$ the predicted error is $9/(12J^2)$.

```
1 set.seed(8); N <- 2e5; xu <- runif(N)
2 t(sapply(c(2, 4, 8, 16), function(J) {
3   b <- cut(xu, seq(0, 1, length.out = J + 1))
4   c(J = J, mse = mean((3*xu - ave(3*xu, b))^2), theory = 9/(12*J^2)) })))
```

```
##           J      mse  theory
## [1,]    2 0.187635 0.18750
## [2,]    4 0.046878 0.04688
## [3,]    8 0.011719 0.01172
## [4,]   16 0.002925 0.00293
```

```
1 fs <- as.numeric(xu > 0.5)
2 c(linear_mse = mean(residuals(lm(fs ~ xu))^2),
3   theory = 0.25*(1 - 3*0.25),
4   two_leaf_tree_mse = mean((fs - ave(fs, xu > 0.5))^2))
```

```
##           linear_mse           theory two_leaf_tree_mse
##           0.0625           0.0625           0.0000
```

On a sample of 400 the first-split gains are noise.

```

1 set.seed(11); q <- 400; z1 <- runif(q); z2 <- runif(q)
2 fx <- as.numeric((z1 > .5) == (z2 > .5)); yx <- fx + rnorm(q, 0, .3)
3 gn <- function(s, v) { i <- if (v == 1) z1 < s else z2 < s
4   sum(i)*sum(!i)/q*(mean(yx[i]) - mean(yx[!i]))^2 }
5 ss <- c(.2, .35, .5, .65, .8)
6 round(rbind(cut = ss, gain_x1 = sapply(ss, gn, v = 1),
7   gain_x2 = sapply(ss, gn, v = 2),
8   percent_of_total_x2 = 100*sapply(ss, gn, v = 2)/
9     sum((yx - mean(yx))^2)), 4)

```

```

##           [,1]  [,2]  [,3]  [,4]  [,5]
## cut      0.2000 0.3500 0.5000 0.6500 0.8000
## gain_x1   0.7533 0.2945 0.0012 0.5719 0.8826
## gain_x2   2.2278 0.9565 0.5922 0.6172 0.3380
## percent_of_total_x2 1.7156 0.7366 0.4560 0.4753 0.2603

```

The largest gain in the table is at $x_2 < 0.2$ and is 1.7% of the total sum of squares, while splitting x_1 at the cut that matters, $\frac{1}{2}$, has the smallest gain of the ten. A greedy tree splits where the noise is.

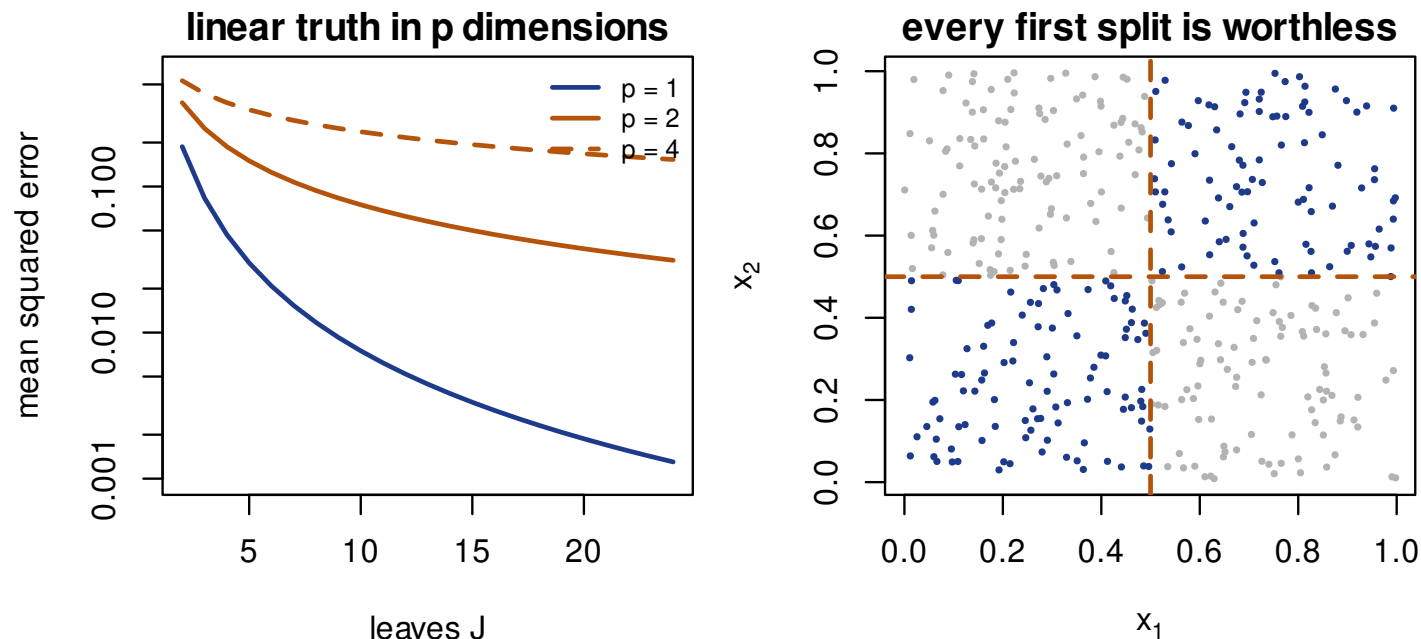


Figure 11.3: Left: Theorem 11.28, the error of the best J -box approximation to a linear truth, on a log scale; the decay $J^{-2/p}$ flattens as the dimension grows. Right: the exclusive-or truth of Theorem 11.30, which four boxes fit exactly and which no single split improves at all.

4.6 Instability (§3.3)

```

1 set.seed(9); Rt <- replicate(200, {
2   i <- sample(nrow(Boston), nrow(Boston)/2)
3   a <- tree(medv ~ ., Boston[i, ]); b <- tree(medv ~ ., Boston[-i, ])
4   c(as.character(a$frame$var[1]), as.character(b$frame$var[1]),
5     sqrt(mean((predict(a, Boston) - predict(b, Boston))^2))) })
6 sort(table(c(Rt[1, ], Rt[2, ])), decreasing = TRUE)

```

```

##
##      rm lstat
## 215    185

```

```

1 c(agree_on_root_variable = mean(Rt[1, ] == Rt[2, ]),
2   rms_prediction_difference = mean(as.numeric(Rt[3, ])),
3   sd_of_medv = sd(Boston$medv))

```

```

##      agree_on_root_variable rms_prediction_difference
##                      0.185                      4.124
##              sd_of_medv
##                      9.197

```

The two halves choose **rm** and **lstat** in nearly equal numbers, agree on the root variable under a fifth of the time, and their predictions on the same observations differ in root mean square by 45% of the standard deviation of the response itself.

4.7 A classification tree end to end (§2)

```

1 set.seed(2); trn <- sample(nrow(Car), 200)
2 tt <- tree(High ~ . - Sales, Car, subset = trn)
3 tab <- table(predicted = predict(tt, Car[-trn, ], type = "class"),
4             observed = Car$High[-trn])
5 tab; c(leaves = sum(tt$frame$var == "<leaf>"),
6       test_error = 1 - sum(diag(tab))/sum(tab))

```

```

##           observed
## predicted  No Yes
##      No  104  33
##      Yes   13  50
##
##  leaves test_error
##    21.00      0.23

```

```

1 set.seed(7); cm <- cv.tree(tt, FUN = prune.misclass)

```

```
2 rbind(size = cm$size, cv_errors = cm$dev)
```

```
##           [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9]
## size         21  19   14    9    8    5    3    2    1
## cv_errors     75  75   75   74   82   83   83   85   82
```

```
1 best <- cm$size[which.min(cm$dev)]
2 pt <- prune.misclass(tt, best = best)
3 tab2 <- table(predicted = predict(pt, Car[-trn, ], type = "class"),
4             observed = Car$High[-trn])
5 c(pruned_leaves = best, test_error = 1 - sum(diag(tab2))/sum(tab2))
```

```
## pruned_leaves    test_error
##           9.000         0.225
```

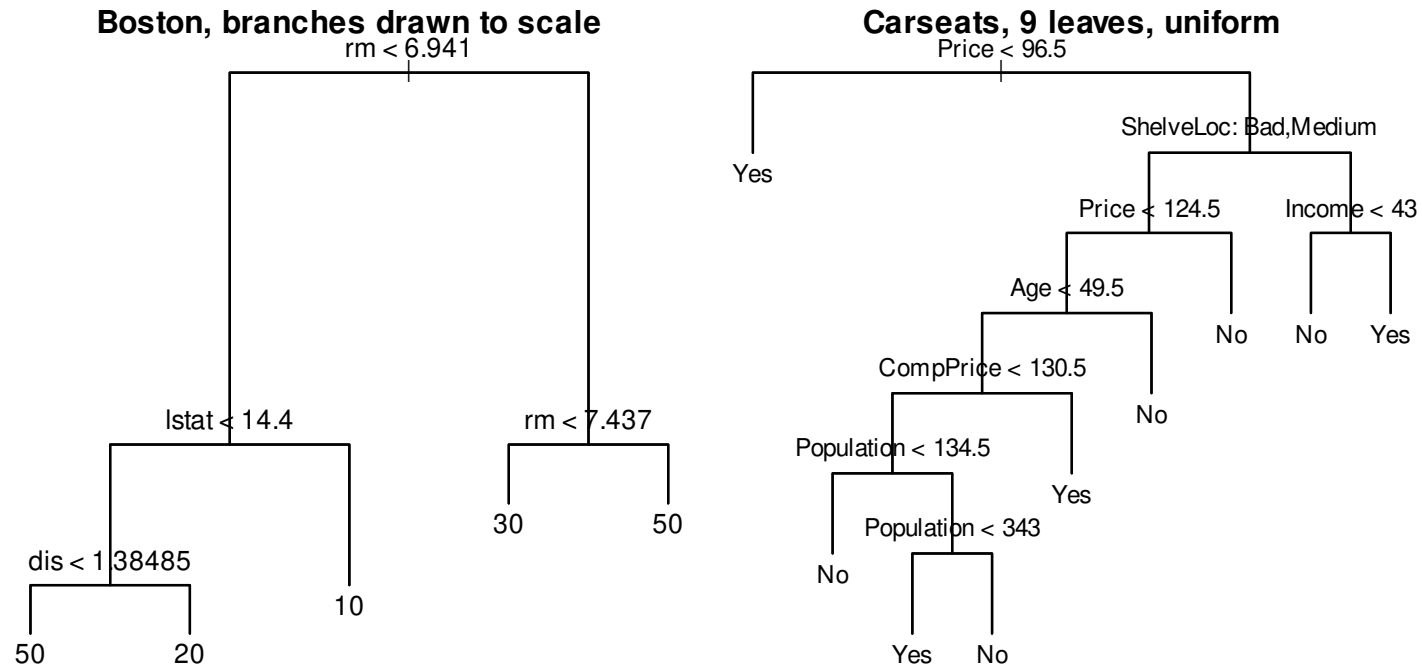


Figure 11.4: Left, **Boston** pruned to five leaves with branch lengths proportional to the drop in the node cost, so a long branch is a split that mattered: the first split on **rm** accounts for most of the tree. Right, the **Carseats** tree of §4.7 drawn with uniform branches, which nine leaves need to stay legible. It splits on **ShelveLoc** directly, with no dummy coding.

5 Pitfalls

5.1 Stopping early instead of pruning

Growing while the gain exceeds a threshold and pruning a large tree are not the same procedure. The exclusive-or truth of Theorem 11.30 is the extreme case: no first split has any population gain at all, so every threshold above zero stops at the root.

```
1 set.seed(15); q <- 800; w1 <- runif(q); w2 <- runif(q)
2 fw <- as.numeric((w1 > .5) == (w2 > .5))
3 dw <- data.frame(w1, w2, y = fw + rnorm(q, 0, .3))
4 t(sapply(c(0.05, 0.01, 0.002, 0), function(md) {
5   g <- tree(y ~ w1 + w2, dw,
6             control = tree.control(q, mindev = md, minsize = 10))
7   c(mindev = md, grown_leaves = sum(g$frame$var == "<leaf>"),
8     mse_of_fit = mean((predict(g, dw) - fw)^2)) })))
```

```
##      mindev grown_leaves mse_of_fit
## [1,]  0.050             1  0.250011
## [2,]  0.010             1  0.250011
## [3,]  0.002            11  0.004584
```

```
## [4,] 0.000 130 0.037224
```

```
1 c(variance_of_f = mean((fw - mean(fw))^2))
```

```
## variance_of_f
```

```
## 0.25
```

At the default `mindev` of 0.01 the tree is the root and its mean squared error is the variance of the target: nothing has been learnt. Growing without a threshold and pruning afterwards does better than either extreme.

```
1 g0 <- tree(y ~ w1 + w2, dw, control = tree.control(q, mindev = 0, minsize = 10))
2 t(sapply(c(2, 4, 8, 16, 40, 130), function(J) {
3   pj <- prune.tree(g0, best = J)
4   c(requested = J, leaves = sum(pj$frame$var == "<leaf>"),
5     mse = mean((predict(pj, dw) - fw)^2)) })))
```

```
## requested leaves mse
## [1,] 2 5 0.052843
## [2,] 4 5 0.052843
## [3,] 8 8 0.002027
## [4,] 16 18 0.008559
```

```
## [5,]      40      48 0.021706
## [6,]     130     130 0.037224
```

Diagnostic. A tree with one leaf on data whose structure is known to be richer, and a fit that improves sharply the moment the stopping rule is loosened. **Fix.** Grow past the point where splits look worthwhile, then prune (§1). The best pruned tree here has mean squared error 0.0020 against the 0.2500 the default stopping rule delivers, a factor of 123, and against 0.0372 for the fully grown tree that was pruned. Both halves are needed: growing finds the structure, pruning removes what the growing added afterwards. Note also that **best** requests a size and returns the nearest one the sequence contains, so 2 and 4 both return five leaves.

5.2 Pruning with the criterion used for growing

Both the error count and the deviance are legitimate node costs (Proposition 11.3), and they select different trees.

```
1 tf <- tree(High ~ . - Sales, Car)
2 t(sapply(1:6, function(s) { set.seed(s)
3   a <- cv.tree(tf, FUN = prune.misclass); b <- cv.tree(tf)
4   c(seed = s, chosen_by_error_count = a$size[which.min(a$dev)],
```

```
5 chosen_by_deviance = b$size[which.min(b$dev)]) }))
```

```
##      seed chosen_by_error_count chosen_by_deviance
## [1,]    1                17                3
## [2,]    2                27                2
## [3,]    3                12                3
## [4,]    4                 7                3
## [5,]    5                27                3
## [6,]    6                27                2
```

Diagnostic. Two runs of the same pipeline reporting trees of two and of twenty-seven leaves. **Fix.** Prune on the loss the model will be judged by. The deviance of Proposition 11.25 is a log likelihood and punishes a confident wrong leaf without limit; the error count of Proposition 11.17 does not. If the tree will be scored by its error rate, use `FUN = prune.misclass`. Note also that the error-count choice is itself unstable across folds, ranging over most of the sequence as the seed changes, which is the instability of §3.3 appearing in the selection step.

5.3 Reading the training curve

```
1 pr <- prune.tree(tb)
2 rbind(leaves = pr$size, training_deviance = round(pr$dev),
```

```
3 cv_deviance = round(cvb$dev))
```

```
##           [,1] [,2] [,3] [,4] [,5] [,6] [,7]
## leaves           9    8    7    6    5    4    3
## training_deviance 6734 7179 7905 9042 10484 13004 16065
## cv_deviance       12767 13071 13350 13442 15607 17788 18590
##           [,8] [,9]
## leaves           2    1
## training_deviance 23377 42716
## cv_deviance       27193 42844
```

Diagnostic. A cost that falls monotonically with the number of leaves, used to choose the number of leaves. **Fix.** The training cost is decreasing in the tree by the refinement property (Proposition 11.3), so its minimiser is always $T_{\max}$ and it carries no information about size. Only the cross-validated column can be minimised. On these data both columns happen to point at nine leaves, which is a coincidence of this example and not a reason to trust the first.

5.4 Cross-validating the pruning but not the growing

```
1 set.seed(21); B <- 40; n <- nrow(Boston)
2 res <- t(replicate(B, {
```

```

3  f <- sample(10, n, replace = TRUE)
4  full <- tree(medv ~ ., Boston)
5  honest <- leaky <- 0
6  for (k in 1:10) {
7    ho <- Boston[f == k, ]
8    h <- prune.tree(tree(medv ~ ., Boston[f != k, ]), best = 5)
9    honest <- honest + sum((ho$medv - predict(h, ho))^2)
10   leaky <- leaky + sum((ho$medv - predict(prune.tree(full, best = 5), ho))^2)
11 }
12 c(honest, leaky)/n })))
13 c(honest_cv_mse = mean(res[, 1]), leaky_cv_mse = mean(res[, 2]),
14    ratio = mean(res[, 2])/mean(res[, 1]))

```

##	honest_cv_mse	leaky_cv_mse	ratio
##	29.6358	20.7186	0.6991

Diagnostic. A cross-validation loop in which only the pruning, or only the choice of α , is redone inside the fold while the splits come from a tree fitted to everything. **Fix.** Refit from the raw data inside each fold, as Definition 11.14 requires. The splits are chosen by looking at the response, so a tree grown on all the data has already seen every held-out observation. The optimism is not small

here: the leaky loop reports 30% less error than the honest one, which would turn a mediocre tree into a good one on paper.

5.5 Reading a single tree as a statement about variables

```
1 set.seed(31); imp <- replicate(200, {  
2   i <- sample(nrow(Boston), replace = TRUE)  
3   as.character(tree(medv ~ ., Boston[i, ])$frame$var[1]) })  
4 round(sort(prop.table(table(imp)), decreasing = TRUE), 3)
```

```
## imp  
##      rm lstat  
## 0.575 0.425
```

Diagnostic. A report that names the root split variable as the most important predictor. **Fix.** Resample and look at the distribution. Here two variables share the root almost equally, and which one appears is a property of the sample and not of the population. Importance measures averaged over many trees are the only form in which the question has a stable answer.

6 Exercises

Exercise 11.1. A node t holds 200 observations with residual sum of squares $R(t) = 4800$. The branch below it has 5 leaves and total residual sum of squares 3000. At what α is the branch collapsed, and what happens to that value if the branch is first pruned to 3 leaves with total residual sum of squares 3600?

Solution. By Corollary 11.13, $\alpha_t = (4800 - 3000)/(5 - 1) = 450$: below 450 the five leaves are worth their price and above it they are not.

With 3 leaves the same formula gives $(4800 - 3600)/(3 - 1) = 600$. The threshold rises because the price is now shared among fewer extra leaves: each of the two remaining splits buys 600 of cost reduction where each of the original four bought 450. This is why the sequence is nested and the thresholds increase (Theorem 11.11): the branches that survive are the ones paying most per leaf.



Exercise 11.2. Explain why the number of pruned subtrees does not have to be enumerated, and give the two properties of h_t that make the threshold in Theorem 11.8 exist and be unique.

Solution. Because R_α adds over the two branches at every internal node, the minimisation splits into independent minimisations at the children, which is the recursion (3). One sweep from the leaves upward therefore evaluates m_α everywhere, and Definition 11.9 reads off the minimiser. $h_t(\alpha) = m_\alpha(t_L) + m_\alpha(t_R) - R(t) - \alpha$ is strictly increasing, because $m_\alpha(t_L) + m_\alpha(t_R)$ is piecewise linear with slopes at least 2 while $R(t) + \alpha$ has slope 1; and $h_t(0) \leq 0$, by the refinement property. A continuous strictly increasing function that starts non-positive and is unbounded above crosses zero exactly once. ■

Exercise 11.3. An `rpart` table reports `CP` = 0.031 at the split that takes a tree from 4 to 5 leaves, and the root node has residual sum of squares 91,200. What is the corresponding α , and what does it mean?

Solution. `rpart` scales by the root cost, so $\alpha = 0.031 \times 91,200 = 2827$. By Corollary 11.13 that is the cost reduction per extra leaf bought by the branch in question: keeping the fifth leaf lowers the residual sum of squares by 2827, and it is worth keeping exactly when the penalty per leaf is below that. The scaling makes `cp` comparable across data sets, since it is the fraction of the root's cost, whereas α carries the units of the response squared. ■

Exercise 11.4. A node holds 500 observations, 300 of class 1 and 200 of class 2. Compute M , G and H . A split sends 250 observations with 200 of class 1 to the left. Compute the three gains and say which criteria would make the split.

Solution. Parent $\mathbf{p} = (0.6, 0.4)$: $M = 0.4$, $G = 1 - 0.36 - 0.16 = 0.48$, $H = -0.6 \ln 0.6 - 0.4 \ln 0.4 = 0.6730$. The ordering $M \leq G \leq H$ of Theorem 11.23 holds, as does $G \leq 2M = 0.8$. Left child (200, 50), $\mathbf{p}_L = (0.8, 0.2)$: $M = 0.2$, $G = 0.32$, $H = 0.5004$. Right child (100, 150), $\mathbf{p}_R = (0.4, 0.6)$: $M = 0.4$, $G = 0.48$, $H = 0.6730$. Both children have 250 observations, so the weighted impurities are the plain averages: M gives $(0.2 + 0.4)/2 = 0.3$, G gives 0.40, H gives 0.5867.

The gains are $0.4 - 0.3 = 0.10$, $0.48 - 0.40 = 0.08$ and $0.6730 - 0.5867 = 0.0863$. All three are positive here, so every criterion would make the split. Note that the modal class changes from 1 to 2 between the children, which is precisely the case Theorem 11.22 excludes, so M is not blind to it. 

Exercise 11.5. Show that a split whose two children keep the parent's modal class has zero misclassification gain, and give the two-class numbers for a parent with (60, 40) split into (35, 15)

and (25, 25).

Solution. Theorem 11.22: if class k^* is modal in all three nodes then $n_t M(\mathbf{p}_t) = n_t - n_{tk^*}$ and the same for each child, and the counts add, so the weighted child value equals the parent's exactly. For the numbers, the parent has $M = 0.40$ and 100 observations, so 40 errors. The left child (35, 15) has modal class 1 and 15 errors; the right child (25, 25) is tied, and either label gives 25 errors. Total 40: the gain is exactly zero. The Gini index sees more: parent 0.48, children 0.42 and 0.50, weighted $0.5(0.42) + 0.5(0.50) = 0.46$, a gain of 0.02. A search on M would refuse this split and a search on G would make it. ■

Exercise 11.6. Show that for two classes the Gini splitting criterion is the regression criterion of Unit 10 applied to a 0–1 response, and say what that implies about a split that isolates five observations.

Solution. Theorem 11.26: coding $y_i \in \{0, 1\}$, a node with proportion $\bar{y}$ has $\sum_i (y_i - \bar{y})^2 = m\bar{y}(1 - \bar{y})$ because $y_i^2 = y_i$, while $G = 2\bar{y}(1 - \bar{y})$, so $mG = 2\sum_i (y_i - \bar{y})^2$ node by node. The two criteria differ by the factor 2 and therefore rank splits identically, and the Gini gain is

$$2n_L n_R (\bar{y}_L - \bar{y}_R)^2 / n_t.$$

The $n_L n_R / n_t$ weight is the consequence. A split isolating five observations out of five hundred has $n_L n_R / n_t = 5(495)/500 = 4.95$, against 125 for an even split, so it must produce a difference in proportions about five times as large to score the same. Purity alone does not buy a split; purity in a leaf large enough to matter does. ■

Exercise 11.7. A predictor has 12 unordered levels. How many splits are there in principle, how many need to be examined, and what has to be true for the reduction to be valid?

Solution. In principle $2^{11} - 1 = 2047$ non-trivial subsets. By Theorem 11.27 it is enough to sort the levels by their mean response and try the 11 initial segments.

The reduction needs a scalar to sort by, so it covers a numeric response and a two-class response, where the class-2 proportion within a level plays that role by Theorem 11.26. With three or more classes there is no such ordering and the argument fails. The proof itself is the two-mean assignment step: with the two fitted values fixed, each level belongs on the side whose value it is closer to, so the optimal subset is everything below the midpoint of the two fitted values. ■

Exercise 11.8. A response is exactly linear in one predictor. How many leaves does a tree need to bring its approximation error below 1% of the error a two-leaf tree makes? Answer the same question when the truth is linear in four predictors.

Solution. By Theorem 11.28 the error is $\beta_1^2/(12J^2)$ in one dimension, so the ratio to $J = 2$ is $4/J^2$. Setting $4/J^2 = 0.01$ gives $J = 20$.

In $p = 4$ dimensions the error is $|\beta|^2/(12J^{1/2})$, so the ratio to $J = 2$ is $(2/J)^{1/2}$ and $(2/J)^{1/2} = 0.01$ needs $J = 20,000$. The exponent is $2/p$, so the number of leaves needed for a fixed accuracy grows like the $p/2$ power of the accuracy demanded. A linear model has no approximation error at all in either case; this is the precise sense in which a tree does not use linear structure. ■

Exercise 11.9. The truth is 1 on two opposite quarters of the unit square and 0 on the others. Can a tree represent it? Will recursive binary splitting find it? What does this say about stopping rules?

Solution. It can: four leaves, split on x_1 at $\frac{1}{2}$ and each half on x_2 at $\frac{1}{2}$ (Theorem 11.30(i)). Recursive binary splitting will not find it. Every axis-aligned split of the whole square leaves both

sides with conditional mean $\frac{1}{2}$, so the population gain is exactly zero (part (iii)); on a sample the gains are pure noise and the first split lands wherever the noise is largest, which §4.5 shows is not at $\frac{1}{2}$.

Hence any rule that stops when the gain is small stops at the root, which is §5.1. Growing past the point where splits look worthwhile and pruning afterwards is the only way the second split ever gets the chance to reveal what the first could not. A linear model does no better here: by part (ii) its best predictor is the constant $\frac{1}{2}$. ■

Exercise 11.10. A cross-validation plot shows the smallest error at the largest tree in the sequence. Is something wrong? What would the one standard error rule do, and why might one prefer it?

Solution. Nothing is wrong. Pruning is a family to search, not an instruction to shrink, and §4.2 finds exactly this on **Boston**: the cross-validated deviance is smallest at all nine leaves. The one standard error rule (Definition 11.15) takes the smallest tree whose cross-validated cost is within one standard error of the minimum. In §4.2 that moves an **rpart** fit from 23 splits to 7. The reason to prefer it is that the cross-validation curve is itself estimated: the differences among

several nearby sizes are often well inside their own standard errors, and among trees that cannot be told apart the smaller one is easier to describe and less variable. ■

Exercise 11.11. Two analysts fit trees to two halves of the same data and report different root variables. Which is right, and what should be reported instead?

Solution. Neither, in the sense the question intends. §4.6 repeats that experiment two hundred times: the two halves agree on the root variable under a fifth of the time, and their predictions differ in root mean square by about 45% of the standard deviation of the response. The root split is the arg max of finitely many gains and a small perturbation reverses the order of the top two (§3.3).

What can be reported is the distribution. Resampling and tabulating the root variable, as in §5.5, shows two candidates sharing it almost equally, which is the honest statement. A stable answer requires averaging over many trees, which is Unit 12. ■

Exercise 11.12. In R: (a) recompute the pruning thresholds from the recursion (3) and match them against `prune.tree`; (b) show that `rpart`'s `cp` is those thresholds divided by the cost of

the root; (c) confirm that sorting the levels of a categorical predictor by their mean response finds the same split as an exhaustive search.

Solution. (a) is §4.1, where the two agree to within 10^{-11} ; (b) is the same section, where the first three thresholds match exactly and the fourth does not, because the two packages grow different trees below that point and prune what they grew; (c) is §4.4, where 127 subsets and 7 ordered candidates reach the same minimum.

The point to watch in (a) is that the recursion is what makes the computation possible at all: the number of pruned subtrees is super-exponential in the depth, while the sweep visits each node once. ■